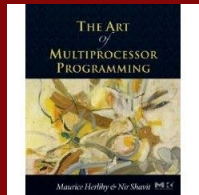
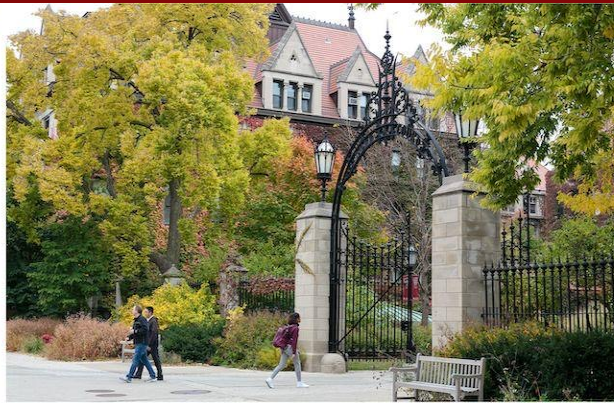


MPCS 52060 - Parallel Programming

M7: Advanced Parallel Patterns and Techniques (Part 1)



Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

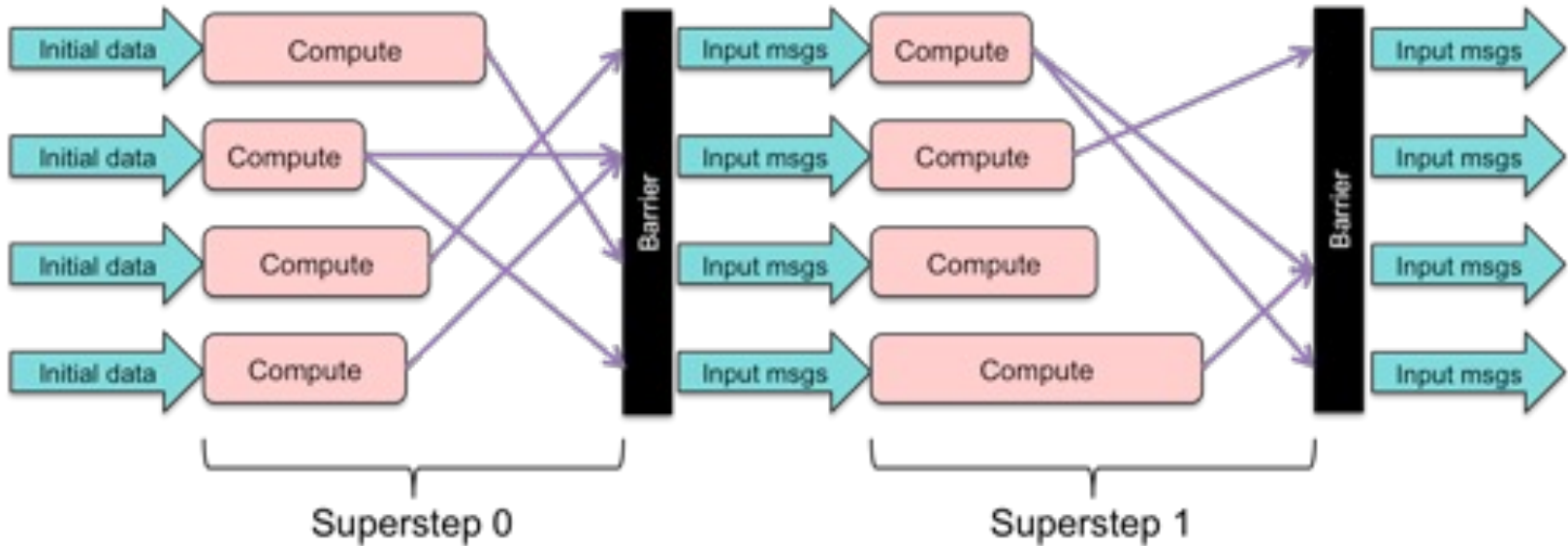
More Go programming patterns

- Fanin
- Select
- Timeout
- Preventing Leaks

Patterns

- BSP (coming up next)
- Fan-out-fan-in
- Pipelines
- Map-reduce

BSP model



<https://people.cs.rutgers.edu/~pxk/417/notes/pregel.html>

Fan-out fan-in

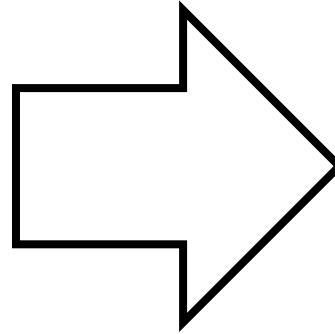
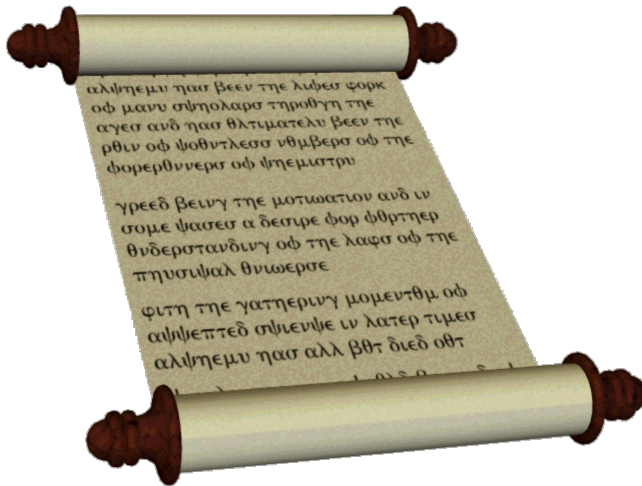
- We discussed fan-in
- If you want to read more, here's a good resource:
- <https://medium.com/geekculture/golang-concurrency-patterns-fan-in-fan-out-1ee43c6830c4>
- There is a fan-out fan-in example in w7_examples on Canvas



Pipelines Demo

Data Parallelism

WordCount



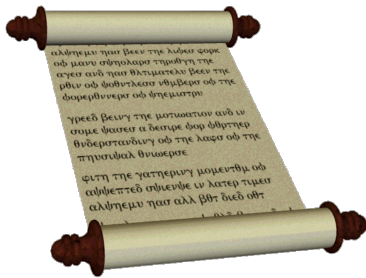
alpha → 8
bravo → 3
charlie → 9
...
zulu → 1

easy to do sequentially ...

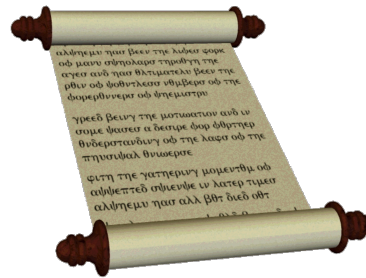
what about in parallel?

MapReduce

split text among *mapping* threads

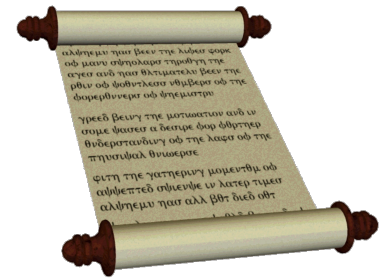


chapter 1



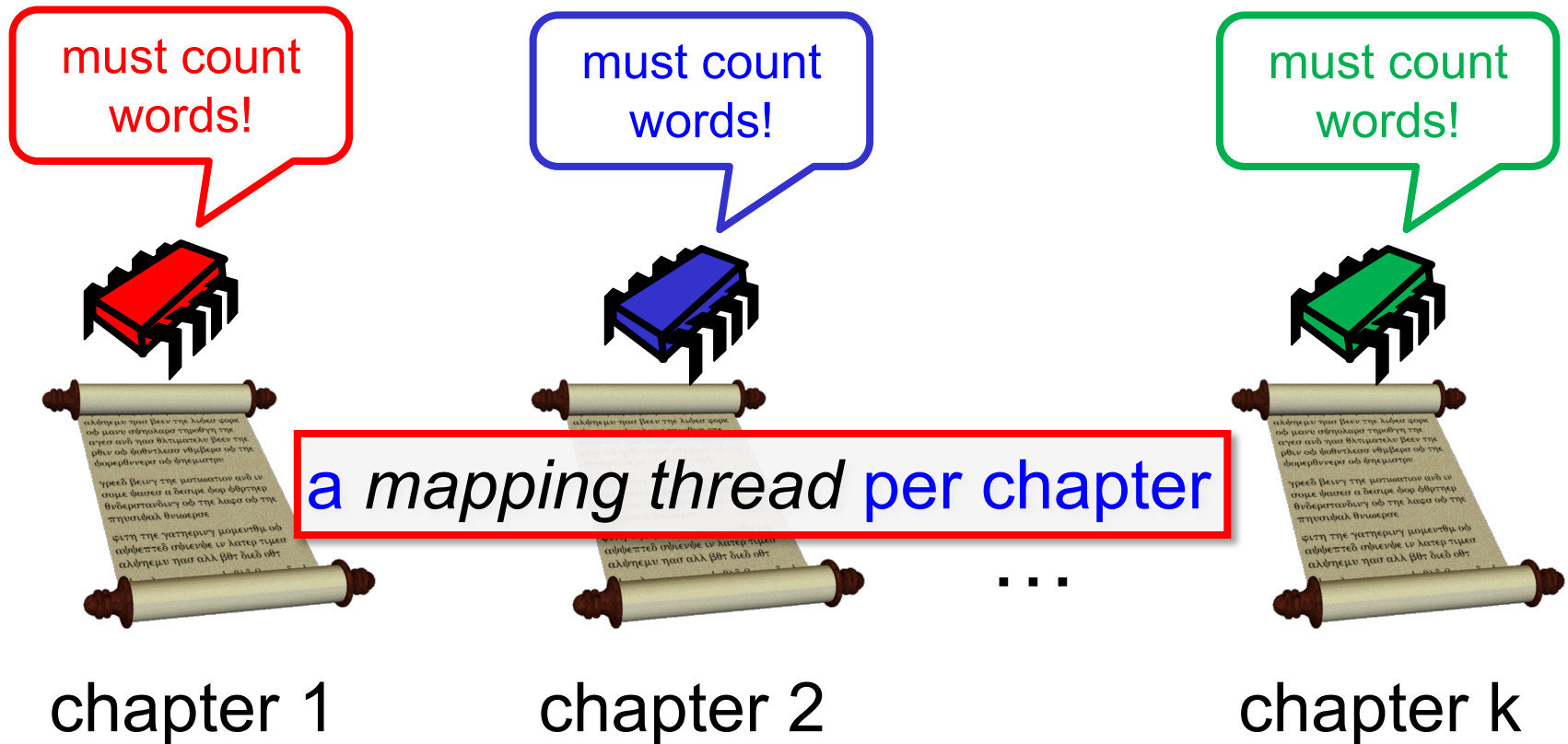
chapter 2

...



chapter k

Map Phase



Map Phase

alpha → 9
juliet → 2,
alpha → 1
tango → 4

each mapper thread produces a *stream* ...
of *key-value* pairs ...

key: word

value: local count



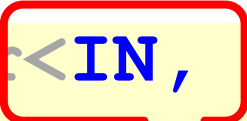
chapter 1

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```



input: document fragment

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```

key: individual word

Mapper Class

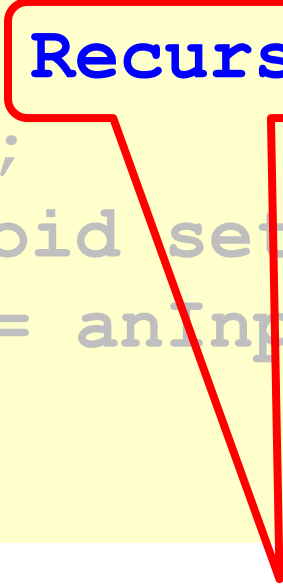
```
abstract class Mapper<IN, K, v>
    extends RecursiveTask<Map<K, v>>> {
    IN input;
    public void setInput(IN anInput) {
        input = anInput;
    }
}
```



value: local count

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```



a task that runs in parallel with other tasks

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```

produces a map: word → count

Mapper Class

```
abstract class Mapper<IN, K, V>  
    extends RecursiveTask<Map<K, V>> {  
    IN input;  
    public void setInput(IN anInput) {  
        input = anInput;  
    }  
}
```

initialize input: which document fragment?

WordCount Mapper

```
class WordCountMapper extends  
  mapreduce.Mapper<  
    List<String>, String, Long  
  > {  
  ...  
}
```

WordCount Mapper

```
class WordCountMapper extends  
  mapreduce.Mapper<  
    List<String>, String, Long  
  > {  
  ...
```

document fragment is
list of words

WordCount Mapper

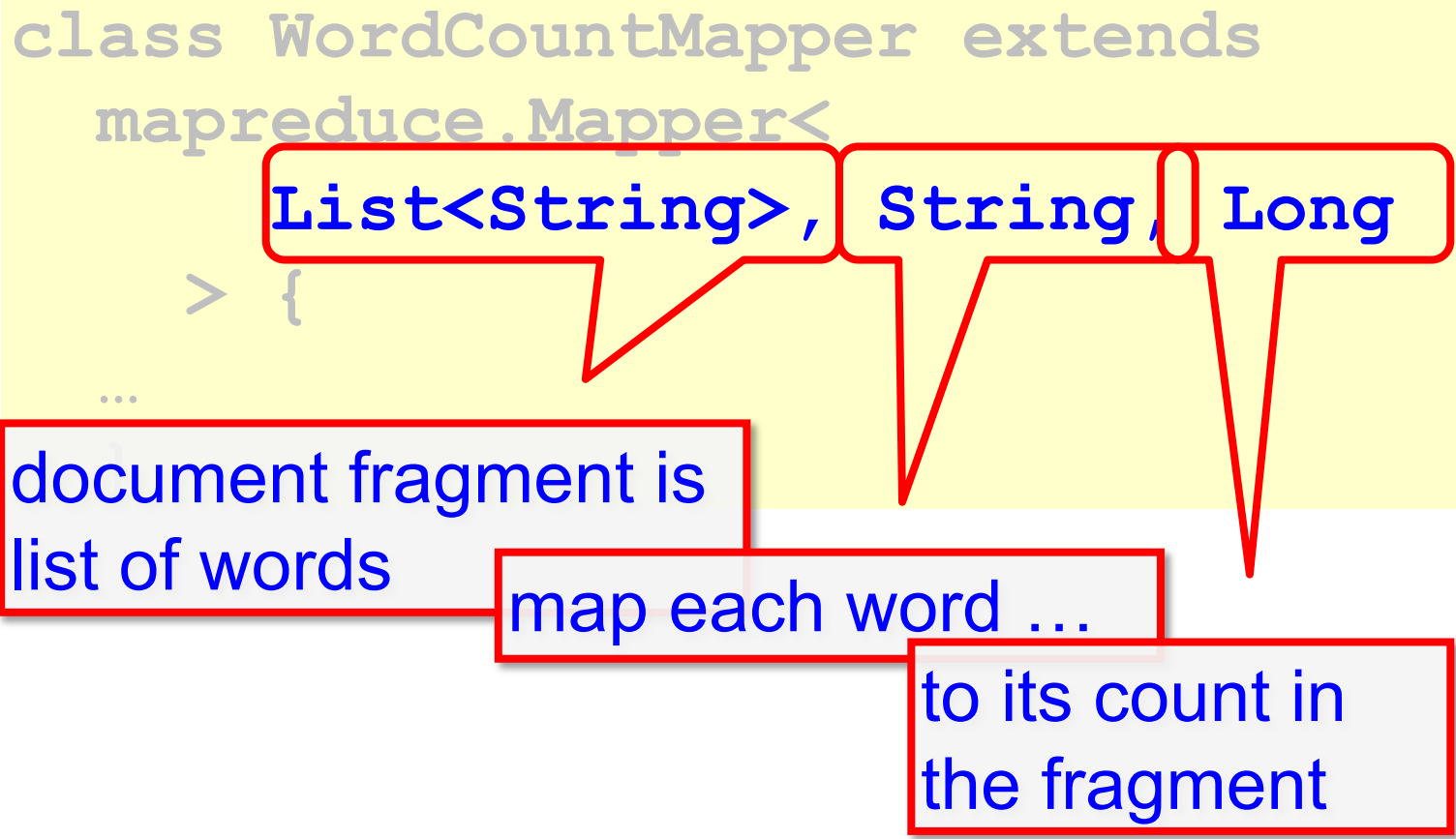
```
class WordCountMapper extends  
  mapreduce.Mapper<  
    List<String>, String, Long  
  > {  
  ...
```

document fragment is
list of words

map each word ...

WordCount Mapper

```
class WordCountMapper extends  
  mapreduce.Mapper<  
    List<String>, String, Long  
  > {  
  ...  
}
```



document fragment is
list of words

map each word ...

to its count in
the fragment

WordCount Mapper

```
Map<String,Long> compute() {  
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
                    1L,  
                    (x, y) -> x + y);  
    }  
    return map;  
}
```

WordCount Mapper

```
Map<String,Long> compute() {
```

```
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
            1L,  
            (x, y) -> x + y);  
    }
```

the `compute()` method constructs the local word count

WordCount Mapper

```
Map<String,Long> compute() {  
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
            1L,  
            (x, y) -> x + y);  
    }  
    return map;  
}
```

create a map to hold the output

WordCount Mapper

```
Map<String,Long> compute() {  
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
                  1L,  
                  (x, y) -> x + y);  
    }  
    return map;  
}
```

examine each word in the document fragment

WordCount Mapper

```
Map<String,Long> compute() {  
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
                   1L,  
                   (x, y) -> x + y);  
    }  
    return map;  
}
```

increment that word's
count in the map

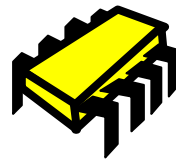
WordCount Mapper

```
Map<String,Long> compute() {  
    Map<String,Long> map = new HashMap<>();  
    for (String word : input) {  
        map.merge(word,  
                  1L,  
                  (x, y) -> x + y);  
    }  
    return map;  
}
```



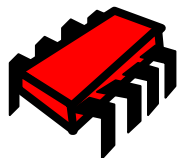
when the local count is complete, return the map

Reduce Phase

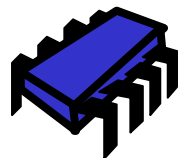


alpha → 4
bravo → 2
...
zulu → 1

a *reducer* thread merges mapper outputs

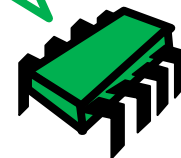


alpha → 2
juliet → 1
tango → 1
...



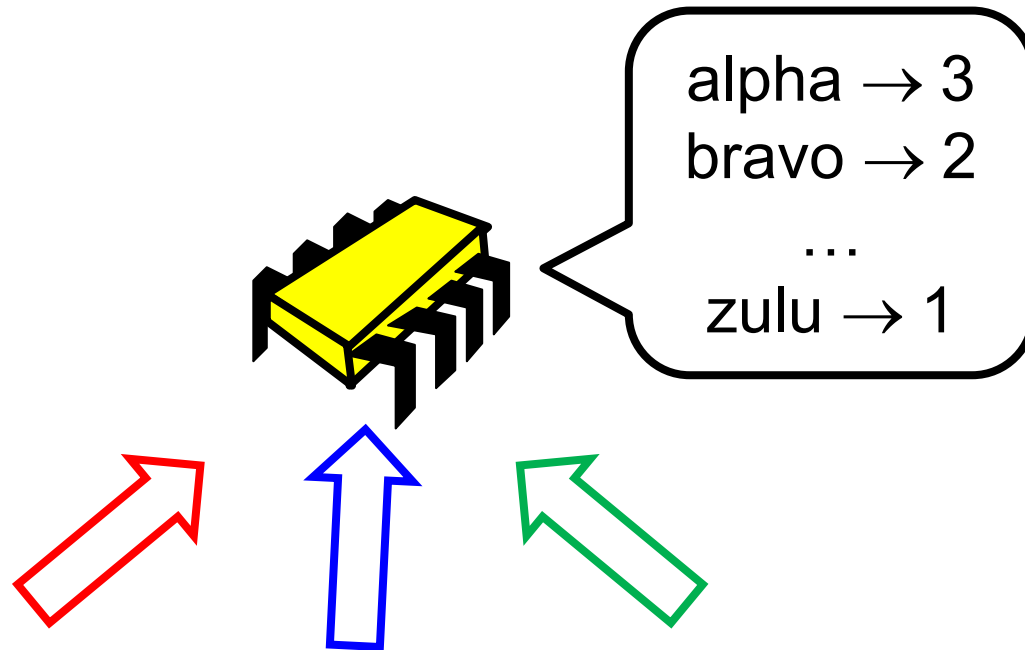
alpha → 1
foxtrot → 1
papa → 1
tango → 1
...

...



alpha → 1
oscar → 1,
bravo → 2...

Reduce Phase



the reducer task produces a *stream* ...

of *key-value* pairs ...

key: word

value: word count

Reducer Class

```
abstract class Reducer<K, V, OUT>
    extends RecursiveTask<OUT> {
    K key;
    List<V> valueList;
    public void setInput(
        K aKey,
        List<V> aList) {
        key = aKey;
        valueList = aList;
    }
}
```

Reducer Class

```
abstract class Reducer<K, V, OUT>
    extends RecursiveTask<OUT> {
    K key;
    List<V> valueList;
    public void setInput(
        K aKey,
        List<V> aList) {
        key = aKey;
        valueList = aList;
    }
}
```

K key;

each reducer is given
a single key (word)

Reducer Class

```
abstract class Reducer<K, V, OUT>  
    extends RecursiveTask<OUT> {  
    K key;  
    List<V> valueList;  
    public void setInput(  
        K aKey,  
        List<V> aList) {  
        key = aKey;  
        valueList = aList;  
    }  
}
```

and a list of associated values
(word count per fragment)

Reducer Class

```
abstract class Reducer<K, V, OUT>
    extends RecursiveTask<OUT> {
    K key;
    List<V> valueList;
    public void setInput(
        K aKey,
        List<V> aList) {
        key = aKey;
        valueList = aList;
    }
}
```

It produces a single summary value
(the total count for that word)

WordCount Reducer

```
static class WordReducer
    extends<String,Long, Long> {
...

    Long compute() {
        Long count = 0;
        for (long c: valueList) {
            Count += c;
        }
        return count;
    }
...
}
```

Pseudocode Implementations

- See `m7/mapreduce/MapReduce.java`
- See `m7/mapreduce/WordCount.java`